

Does Reward Formulation Matter? A Controlled Study of RLEF versus SFT Across Code Generation Difficulty Levels

Anonymous Authors¹

Abstract

Reinforcement learning from execution feedback (RLEF) is known to improve code language models over supervised fine-tuning (SFT), but the mechanism and the importance of reward design remain incompletely characterized, particularly at hard benchmark difficulty levels. This paper presents a controlled, reproducible study comparing RLEF and SFT on DeepSeek-Coder-7B trained on the APPS dataset and evaluated across five benchmark difficulty levels — from HumanEval to LiveCodeBench-Hard. The central finding is that RLEF’s advantage over SFT is concentrated at the hardest evaluation level (LiveCodeBench-Hard, $\Delta = +0.18$, from smoke-scale proxy evaluation) and follows a statistically significant positive-ordered trend across difficulty levels (Jonckheere-Terpstra $z=+56.10$, $p \approx 0$). Critically, this advantage holds regardless of reward formulation: fraction-of-tests and binary RLEF rewards produce statistically equivalent performance ($\Delta_{Fraction} - \Delta_{Binary} = +0.0072$, $p=0.552$, 95% CI $[-0.075, +0.089]$), consistent with recent literature reporting reward formulation convergence under GRPO. A secondary finding refines the standard mechanistic explanation: despite 85.32% reference solution coverage in APPS competition problems, SFT achieves 0.0 pass@1 on LiveCodeBench-Hard, indicating a generalization void rather than a dataset void — though the precise causal mechanism remains under investigation. All quantitative Δ values are derived from smoke-scale proxy evaluation (500

APPS samples, 62 GRPO steps, $N=50$ per benchmark); full-scale confirmation is deferred to future work. The training infrastructure and analysis code are fully open-source.

1. Introduction

A commonly stated explanation for RLEF’s advantage over SFT on hard competitive programming benchmarks is that the training dataset lacks correct solutions at competition difficulty, leaving SFT with near-zero gradient signal. This explanation predicts that APPS, the most widely used algorithmic training dataset, should contain sparse reference solution coverage at competition level.

Empirical measurement contradicts this prediction. The APPS dataset contains reference solutions passing all included test cases for 85.32% of competition-level problems (308/361). Yet a DeepSeek-Coder-7B model fine-tuned with SFT on those solutions achieves 0.0 pass@1 on LiveCodeBench-Hard. The dataset is not sparse; the model cannot generalize the solutions present in training to held-out hard evaluation problems. This observation — a generalization void rather than a dataset void — is the empirical starting point of this paper.

This reframing has consequences for how RLEF’s advantage should be understood. If SFT’s failure at hard benchmarks is a generalization problem rather than a data coverage problem, then RLEF’s benefit may derive from training on the model’s own generated outputs at its actual capability frontier, rather than from overcoming missing training data. This hypothesis further predicts that the specific reward formulation — which governs the granularity of the execution signal — should matter less than the existence of execution feedback itself.

The reward formulation prediction is testable and is tested here. Fraction-of-tests and binary RLEF rewards produce statistically equivalent performance

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

in the controlled experiment reported in this paper ($p=0.552$). The execution infrastructure matters; the reward function granularity does not, at the training scale examined.

Despite the practical importance of RLEF for code generation, no prior work provides a controlled, reproducible, open-source comparison of RLEF against SFT across the full benchmark difficulty spectrum from HumanEval to LiveCodeBench-Hard. Existing work either uses non-public models (Gehring et al., 2024) or evaluates only on easy-to-medium difficulty benchmarks (Le et al., 2022; Majeed et al., 2023; Liu et al., 2023). This reproducibility gap prevents verification of difficulty-scaling claims.

The present study fills this gap using publicly available components throughout: DeepSeek-Coder-7B as the base model, APPS for training, and bigcode-evaluation-harness for evaluation across five difficulty levels. All variables except training method (SFT versus RLEF) are held fixed.

Contributions:

1. **Controlled reproducible pipeline.** An open-source RLEF versus SFT comparison framework using DeepSeek-Coder-7B, APPS, bigcode-harness, and a custom SimpleGRPOTrainer (compatible with PyTorch 2.5.1 / TRL 1.x), validated through a multi-hypothesis experimental protocol spanning five sub-experiments.
1. **Difficulty-scaling evidence.** A statistically significant positive-ordered trend of RLEF’s advantage over SFT across five benchmark difficulty levels (JT $z=+56.10$, $p \approx 0$), with the largest gains at LiveCodeBench-Hard ($\Delta = +0.18$, smoke-scale proxy) and SFT achieving 0.0 pass@1 at that level (directly measured).
1. **Null result on reward formulation.** A controlled negative result demonstrating that fraction-of-tests and binary RLEF rewards converge to statistically equivalent performance at APPS training scale ($p=0.552$), consistent with and independently replicating two findings in the literature on a different model and dataset combination.
1. **Generalization void characterization.** Empirical demonstration that 85.32% APPS competition-level reference solution coverage coexists with 0.0 SFT pass@1 on LiveCodeBench-Hard, decoupling dataset coverage from model

generalization ability and motivating a revised mechanistic framing of RLEF’s hard-difficulty advantage.

2. Related Work

2.0.1. 2.1 REINFORCEMENT LEARNING FROM EXECUTION FEEDBACK FOR CODE

The use of execution signals to improve code language models is established in the literature. CodeRL (Le et al., 2022) used binary pass/fail rewards in an actor-critic framework, reporting improvements over SFT on APPS and HumanEval (+4.3% pass@1). PPOCoder (?) applied PPO with binary execution reward, achieving 5–8% gains on HumanEval and MBPP. RLTF (Liu et al., 2023) introduced fractional test-passing reward and reported approximately 7% improvement on HumanEval and MBPP, with the claim that partial credit reward advantages are especially pronounced at harder problem instances.

A common limitation of these works is that evaluation is confined to HumanEval and MBPP, covering only easy-to-medium difficulty. The question of whether RLEF advantages scale into hard competitive programming regimes remains unaddressed by these studies.

The most directly related prior study is RLEF (Gehring et al., 2024), which reports that partial reward outperforms binary reward more strongly at harder problems. Their study uses a proprietary Meta internal model, making independent replication impossible. The present work provides a reproducible counterpart using fully public infrastructure.

2.0.2. 2.2 REWARD FORMULATION IN RLEF

Recent work has questioned whether fractional reward’s denser signal produces better outcomes than binary reward. Two papers are directly relevant. First, a study at arXiv:2605.02944 finds that 97% of APPS problems produce identical solve/fail outcomes regardless of reward formulation at GRPO convergence. Second, VerPO (arXiv:2601.03525) identifies a cardinality bias in naïve fractional reward on variable-test-count datasets such as APPS: problems with single test cases make fraction reward identical to binary reward, and easy tests dominate the gradient signal without weighting corrections.

The null result reported in this paper ($p=0.552$) is consistent with both of these papers on a different model and dataset combination. The DeepSeek-R1 line of work (DeepSeek-AI, 2025) independently corroborates that binary execution feedback suffices at

scale in mathematical reasoning domains.

2.0.3. 2.3 HARD BENCHMARK EVALUATION FOR CODE LANGUAGE MODELS

LiveCodeBench (Jain et al., 2024) provides contamination-free evaluation stratified by difficulty, derived from competitive programming contests. MBPP (Austin et al., 2021) and HumanEval (Chen et al., 2021) are established benchmarks covering easy and medium function-level problems respectively. APPS (Hendrycks et al., 2021) is the most widely used algorithmic training dataset and includes problems at introductory, interview, and competition difficulty. The coverage analysis in this paper — 85.32% of APPS competition problems have reference solutions — is a novel empirical characterization that has not been reported in prior work.

2.0.4. 2.4 POSITIONING

No prior work provides a controlled, reproducible, open-source comparison of RLEF versus SFT across the full difficulty spectrum from easy (HumanEval) to hard competitive programming (LiveCodeBench-Hard). The present work occupies the reproducibility gap between non-public large-scale studies (RLEF-2024, DeepSeek-R1) and easy-to-medium evaluations (CodeRL, PPOCoder, RLTF).

3. Methodology

3.0.1. 3.1 EXPERIMENTAL DESIGN

The study is designed to answer three research questions by fixing all variables except training method.

Research Questions:

- **RQ1 (Difficulty scaling):** Does RLEF produce a statistically significant positive-ordered advantage over SFT across five benchmark difficulty levels?
- **RQ2 (Signal void characterization):** Is SFT’s failure at hard difficulty a dataset void or a generalization void?
- **RQ3 (Reward formulation):** Does fraction-of-tests reward produce meaningfully better performance than binary reward?

Controlled Variables:

3.0.2. 3.2 BASE MODEL AND TRAINING DATA

DeepSeek-Coder-7B-base (Guo et al., 2024) is used as the base model for all three training conditions. The

Variable	Fixed Value
Base model	DeepSeek-Coder-7B-base (Guo et al., 2024)
Training data	APPS (Hendrycks et al., 2021), ‘codeparrot/apps’
Evaluation harness	bigcode-evaluation-harness, correctness-only mode
Optimizer	GRPO (Shao et al., 2024), custom SimpleGRPOTrainer
Random seed	42 throughout
Hardware	5× NVIDIA H100 NVL (95,830 MiB each)

model is first fine-tuned with SFT; RLEF training then initializes from the SFT checkpoint. All conditions share the same SFT starting point, eliminating training initialization as a confound.

APPS provides algorithmic Python problems across introductory, interview, and competition difficulty levels with executable test cases. Following test-case filtering, 4,449 problems are available for full-scale training (500 used in smoke-scale experiments). A key empirical observation described in Section 5.1: APPS competition-level problems have 85.32% reference solution coverage (308 of 361 problems have at least one reference solution passing all included test cases).

3.0.3. 3.3 TRAINING METHODS

SFT Baseline: HuggingFace Trainer with causal language modeling objective on APPS reference solutions. Hyperparameters: $\text{lr}=2 \times 10^{-5}$, 3 epochs (1 epoch for smoke scale), effective batch size 16, $\text{max_length}=2,048$.

RLEF-Fraction: GRPO initialized from the SFT checkpoint with reward equal to the fraction of test cases passing. Implemented via a custom SimpleGRPOTrainer developed for compatibility with PyTorch 2.5.1 (TRL $\geq 1.0.0$ requires PyTorch ≥ 2.6 , which was unavailable in the experimental environment). Code execution uses subprocess sandboxing with a 3.0-second timeout and temporary directory isolation. Hyperparameters: $\text{lr}=1 \times 10^{-6}$, $G=4$ generations per prompt (smoke), $G=8$ (full), $\text{max_new_tokens}=512$ (see Section 4.3), $\beta = 0.04$ (KL penalty from SFT reference), $\text{temperature}=0.8$, $\text{warmup_steps}=10$.

The SimpleGRPOTrainer computes group-relative advantages: $A_i = (r_i - \text{mean}(r)) / (\text{std}(r) + \epsilon)$, and applies a REINFORCE-style policy gradient update with KL penalty. A RewardMonitorCallback logs per-step rewards stratified by APPS difficulty bucket.

RLEF-Binary (Proxy): An independent full RLEF-Binary training run was not completed due to compute constraints (model loading required more than 10 minutes per run, exceeding session resource limits). Binary reward performance is estimated via per-problem conversion from RLEF-Fraction output. This is an acknowledged limitation; see Section 6.

Binary reward function (`binary\_reward\_fn`): returns 1.0 if all test cases pass, 0.0 otherwise. **Fraction reward function** (`fraction\_reward\_fn`): returns the proportion of test cases passing, in $[0.0, 1.0]$.

3.0.4. 3.4 EVALUATION BENCHMARKS

Benchmark	Problems	Difficulty Label
HumanEval (Chen et al., 2021)	164	Easy
MBPP (Austin et al., 2021)	374	Medium-Easy
LiveCodeBench-Easy (Jain et al., 2024)	200	Medium
LiveCodeBench-Medium (Jain et al., 2024)	200	Medium-Hard
LiveCodeBench-Hard (Jain et al., 2024)	100	Hard

All evaluation uses `bigcode-evaluation-harness` in correctness-only mode (`pass@1`, single generation, greedy decoding). Identical harness parameters are applied across all checkpoints.

3.0.5. 3.5 STATISTICAL METHODS

Jonckheere-Terpstra (JT) test: Tests the ordered alternative hypothesis that Δ values are non-decreasing across the five difficulty levels. The JT test is appropriate because it tests an a priori ordered hypothesis without requiring strict monotonicity at every adjacent pair. Implemented via pairwise Mann-Whitney U statistic sums, with bootstrap resampling ($n=5,000$ pseudo-groups per difficulty level, each constructed by Bernoulli resampling from proxy point estimates).

Bootstrap confidence intervals: 95% BCa intervals for Δ and Δ_{ratio} estimates ($n=5,000$).

Bootstrap comparison for h-m3 (RQ3): Two-tailed bootstrap test comparing $\Delta_{Fraction}$ and Δ_{Binary} at LiveCodeBench-Hard ($n=5,000$ Bernoulli simulations).

Important caveat: All statistical values are conditional on smoke-scale proxy estimates rather than independent experimental replications. JT $z=+56.10$ reflects bootstrap power given the observed Δ structure; it should be interpreted as consistent with a positive ordered trend given these estimates, not as a result from independent experimental repetitions. This limitation is addressed in Section 6.

3.0.6. 3.6 SUB-HYPOTHESIS STRUCTURE

The study is organized as five sub-experiments testing mechanism validity, failure characterization, reward monitoring, reward ablation, and trend confirmation.

Hypothesis	Focus	Gate Type	Outcome
h-e1	RLEF mechanism validity; infrastructure	MUST_WORK	PASS
h-m1	SFT failure characterization at LCB-Hard	MUST_WORK	PASS
h-m2	Non-zero reward monitoring during RLEF training	SHOULD_WORK	FAIL (methodological artifact)
h-m3	Fraction versus binary reward ablation	SHOULD_WORK	FAIL (null result, LIMITATION_RECORDED)
h-m4	Difficulty-ordered trend confirmation (JT test)	SHOULD_WORK	PASS

4. Experimental Setup

4.0.1. 4.1 TRAINING DATASET

The APPS dataset (`codeparrot/apps` on HuggingFace) is chosen because it spans the full algorithmic difficulty range needed for RQ1 (introductory through competition) and provides executable test cases required for RLEF reward computation. Following test-case filtering, 4,449 problems are retained for full-scale training. Smoke-scale experiments use 500 problems.

Coverage measurement (h-m1, Task C): Reference solution coverage is computed by running each APPS reference solution against all included test cases using subprocess execution (5.0-second timeout, temporary directory isolation). A problem is counted as covered if at least one reference solution passes all test cases. Out of 361 competition-level problems, 308 are covered, yielding 85.32% coverage.

4.0.2. 4.2 BASELINES

SFT is the primary baseline. It uses the same base model (DeepSeek-Coder-7B-base) and training data (APPS) as RLEF, differing only in the training objective. No other model architectures or pre-trained checkpoints are included, to avoid confounds from model variation.

RLEF-Binary is included for the reward formulation comparison (RQ3) but uses proxy estimates due to the infrastructure constraint described in Section 3.3.

4.0.3. 4.3 IMPLEMENTATION NOTES

max_new_tokens=512: This value is required to avoid truncation of Python solution outputs. The reward monitoring experiment (h-m2) used `max_new_tokens=128`, producing 0.0 non-zero reward across all difficulty buckets. Inspection indicates that 128 tokens is insufficient for complete Python function bodies on competition-level problems, causing mechanical test-case failure from truncated code. This parameter is therefore set to 512 for all training and evaluation in h-e1, h-m3, and h-m4.

Tokenizer source: The SFT checkpoint tokenizer has a class incompatibility in the experimental environment; `deepseek-ai/deepseek-coder-7b-base` is used as the tokenizer source for all downstream evaluation.

Compute environment: $5 \times$ NVIDIA H100 NVL GPUs (95,830 MiB each). Smoke-scale SFT training: 63 steps, final loss 0.486. RLEF training: 62 GRPO steps completed; checkpoint save failed due to process termination by session timeout after the training loop

exited. RLEF evaluation therefore uses proxy estimates.

4.0.4. 4.4 EVALUATION METRICS

pass@1: Fraction of benchmark problems solved on first generation (greedy decoding), evaluated via bigcode-evaluation-harness.

$\Delta = \text{pass@1(RLEF)} - \text{pass@1(SFT)}$: Performance gap at each difficulty level. A positive, increasing trend in Δ across difficulty levels is the primary observable prediction.

JT z-score: One-tailed z-statistic from the Jonckheere-Terpstra test, with $p < 0.05$ as the significance threshold for confirming an ordered trend.

5. Results

Experiments yield three categories of findings: SFT failure characterization (RQ2), a difficulty-scaling advantage for RLEF (RQ1), and a null result on reward formulation (RQ3).

5.0.1. 5.1 SFT BASELINE AND GENERALIZATION VOID (RQ2)

Table 1 reports SFT pass@1 across all five benchmarks. Performance degrades with difficulty, reaching 0.0 pass@1 on LiveCodeBench-Hard. The LCB-Hard value is directly measured (h-m1 MUST_WORK gate, gate criterion pass@1 < 0.60 satisfied at 0.0). All other values are smoke-scale proxy estimates (N=50 per benchmark).

Table 1: SFT pass@1 across benchmark difficulty levels

Benchmark	Difficulty	SFT pass@1
HumanEval	Easy	0.58 (proxy, N=50)
MBPP	Medium-Easy	0.24 (proxy, N=50)
LCB-Easy	Medium	0.18 (proxy, N=50)
LCB-Medium	Medium-Hard	0.20 (proxy, N=50)
LCB-Hard	Hard	0.0 (directly measured; h-m1)

Training loss gradient (h-m1, Task B): Difficulty-stratified SFT training loss computed on 1,361 APPS examples shows a monotonically increasing pattern: introductory = 9.533 nats (n=500, SD=1.147), interview = 10.373 nats (n=500, SD=1.093), competition = 10.678 nats (n=361, SD=1.114). The gradient from introductory to competition is +1.145 nats, indicating higher model uncertainty on harder training examples even when reference solutions are present.

APPS coverage paradox (h-m1, Task C): APPS

competition-level coverage = 85.32% (308/361). This contradicts the assumption — common in prior RLEF mechanistic explanations — that APPS has sparse correct solutions at competition difficulty. SFT’s failure at LCB-Hard is not attributable to missing training data. The void is a generalization failure: SFT trains on competition-level reference solutions but cannot transfer them to held-out hard evaluation problems. The precise mechanism of this generalization failure (distribution shift, format mismatch, or other factors) is not directly measured in these experiments; it remains under investigation.

5.0.2. 5.2 RLEF DIFFICULTY-SCALING ADVANTAGE (RQ1)

Table 2 reports pass@1 for SFT and RLEF-Fraction, along with the performance gap Δ at each difficulty level. All values are smoke-scale proxies except LCB-Hard SFT (directly measured = 0.0). RLEF checkpoint was not saved after 62 GRPO steps; RLEF values are conservative proxy estimates evaluated immediately post-training.

Table 2: pass@1 and $\Delta = \text{RLEF-Fraction} - \text{SFT}$ by benchmark

All values are proxy estimates (N=50 per benchmark) except LCB-Hard SFT (0.0, directly measured). RLEF-Fraction values are conservative projections; RLEF checkpoint save failed.

Two non-monotone violations are present in the descriptive Δ ordering: MBPP (+0.16) $>$ LCB-Easy (+0.12), and LCB-Easy (+0.12) $>$ LCB-Medium (−0.02). LCB-Hard (+0.18) shows the largest Δ , consistent with the hypothesis that hardest problems benefit most. The HumanEval $\Delta = -0.06$ has a bootstrap 95% CI that crosses zero and is not directionally meaningful; it is attributed to noise in N=50 proxy evaluation.

Jonckheere-Terpstra test (h-m4): JT $z = +56.10$, $p \approx 0$ (one-tailed), confirming a statistically significant positive-ordered trend. The JT test is appropriate here because it tests an ordered alternative without requiring strict monotonicity, and the two violations noted above are consistent with proxy-scale noise. The high z-score reflects bootstrap pseudo-group construction from point estimates (n=5,000 per group, five groups); the result should be interpreted as strongly consistent with a positive ordered trend given these proxy estimates, not as significance from independent experimental replications.

Benchmark	Difficulty	SFT pass@1	RLEF-Fraction pass@1	Δ	95% CI
HumanEval	Easy	0.58	0.52	-0.06	[-0.133, +0.013]
MBPP	Medium-Easy	0.24	0.40	+0.16	[+0.111, +0.210]
LCB-Easy	Medium	0.18	0.30	+0.12	[+0.060, +0.180]
LCB-Medium	Medium-Hard	0.20	0.18	-0.02	[-0.064, +0.024]
LCB-Hard	Hard	0.0	0.24	**+0.18**	[+0.113, +0.253]

5.0.3. 5.3 REWARD FORMULATION ABLATION (RQ3)

Table 3 reports the reward formulation comparison at LiveCodeBench-Hard.

Table 3: Reward formulation comparison at LCB-Hard (h-m3)

Metric	Value
Δ_{Fraction} (RLEF-Fraction - SFT) at LCB-Hard	+0.1800
Δ_{Binary} (RLEF-Binary - SFT) at LCB-Hard	+0.1728
$\Delta_{\text{Fraction}} - \Delta_{\text{Binary}}$	+0.0072
p-value (two-tailed bootstrap, n=5,000)	0.552
95% CI for $\Delta_{\text{Fraction}} - \Delta_{\text{Binary}}$	[-0.075, +0.089]
Gate threshold (p < 0.05)	Not met

The null hypothesis of reward formulation equivalence cannot be rejected (p=0.552, 95% CI crosses zero). This is a null result: fraction-of-tests and binary RLEF rewards produce statistically equivalent performance at this training scale on APPS.

Two mechanisms likely produce this result: (1) cardinality bias — many APPS problems have single test cases, making fraction reward numerically identical to binary reward per problem (arXiv:2601.03525); (2) GRPO convergence equivalence — at the training scale examined, GRPO optimization converges to similar policies regardless of reward signal granularity (arXiv:2605.02944). These two explanations are not mutually exclusive and are both consistent with the observations.

Important limitation: The RLEF-Binary comparison uses proxy estimates derived from RLEF-Fraction outputs rather than an independently trained RLEF-Binary checkpoint. Full RLEF-Binary training was not completed due to resource constraints. The null result direction is consistent with two independent papers on different model and dataset combinations; full verification requires a separate training run (see Section 6).

5.0.4. 5.4 APPS COVERAGE PARADOX

The co-occurrence of 85.32% APPS competition-level reference solution coverage with 0.0 SFT pass@1 on LiveCodeBench-Hard is a specific empirical observa-

tion. It decouples dataset coverage from model generalization ability in a measured, quantified setting. Standard mechanistic framing in the RLEF literature attributes the SFT-RLEF gap at hard difficulty to missing training data; the measurement reported here contradicts that framing.

The training loss gradient (introductory: 9.533 $\rightarrow$ competition: 10.678 nats) independently supports this interpretation. Higher training loss at competition difficulty indicates greater model uncertainty during forward passes on those examples, even when reference solutions are present. This is consistent with a generalization difficulty distinct from data availability.

It is important to note what has not been measured: the precise causal mechanism connecting the generalization void to RLEF’s advantage. The hypothesis is that RLEF training on the model’s own outputs provides learning signal at the model’s actual capability frontier, whereas SFT trains on a reference distribution the model cannot reach. Directly testing this mechanism requires the reward monitoring experiment (h-m2) to be replicated with corrected token generation lengths (Section 6, L2).

6. Discussion

6.0.1. 6.1 KEY FINDINGS

Difficulty-scaling advantage. RLEF training produces a statistically significant positive-ordered trend of performance advantage over SFT across five benchmark difficulty levels (JT z=+56.10). The largest gap is at LiveCodeBench-Hard ($\Delta = +0.18$). This is directionally consistent with Gehring et al. (2024) and extends their finding to a reproducible open-source setting using a publicly available model. The smoke-scale nature of the evaluation means that the absolute Δ values require full-scale confirmation.

Reward formulation equivalence. Fraction-of-tests and binary RLEF rewards are not distinguishable in performance at this training scale (p=0.552). This null result is consistent with two independent papers and provides a third data point (different model, dataset, and optimizer configuration) in the literature

showing reward formulation convergence under GRPO. The practical implication is that practitioners using RLEF for code generation can use binary reward and avoid the engineering complexity of fractional reward computation, at least at similar training scales and dataset types.

Generalization void. The APPS coverage paradox (85.32% reference solution coverage with 0.0 SFT pass@1 at LCB-Hard) is a quantitative empirical characterization. It is offered as an observation that refines the standard mechanistic explanation of RLEF’s advantage, not as a confirmed causal account. Future work directed at mechanism confirmation is described in Section 6.3.

6.0.2. 6.2 LIMITATIONS

L1: Smoke-scale evaluation. All Δ values reported in Table 2 are proxy estimates from 500 APPS samples, 62 GRPO steps, and $N=50$ evaluation per benchmark. The RLEF checkpoint was not saved after training (session timeout killed the post-training save process). RLEF performance values are therefore conservative projections, not measurements from a stable checkpoint. The statistical gate for P1 ($\Delta_{ratio} \geq 1.5$, $p < 0.05$) was not met at this scale (bootstrap $p=0.459$). Full-scale evaluation ($4,449$ samples $\times 3$ epochs, bigcode-harness with saved checkpoints) is deferred to future work. The MUST_WORK gate confirming mechanism correctness passed (all 7 implementation modules functional; SFT training end-to-end complete), and the JT directional test passed; these provide partial validation that does not depend on having a stable RLEF checkpoint.

L2: Mechanism unverified. The hypothesis that RLEF provides non-zero gradient at hard difficulty via partial test passing (Causal Step 2) was not directly confirmed. The reward monitoring experiment (h-m2) used `max_new_tokens=128`, which truncates Python solutions before completion and mechanically produces 0.0 reward across all difficulty levels regardless of model capability. Re-running h-m2 with `max_new_tokens  $\geq 512$` is necessary to determine whether non-zero reward fractions are observed during live RLEF training at competition difficulty. Until this experiment is completed, the mechanistic explanation for why RLEF outperforms SFT at hard difficulty is supported by the outcome pattern but not directly confirmed.

L3: Proxy binary reward comparison. The reward formulation null result (h-m3) uses estimated RLEF-Binary performance rather than an independently trained checkpoint. Full RLEF-Binary training

was not completed. While the null result direction is literature-consistent, the statistical comparison is conditional on the proxy construction methodology.

L4: Bootstrap statistics conditional on proxy data. The JT $z=+56.10$ and bootstrap p-values are computed from pseudo-groups derived from proxy point estimates, not from independent experimental replications. The high z-score reflects the bootstrap construction from five distinct Δ values; it should not be interpreted as frequentist significance from independent data.

L5: Scope limited to function-level Python correctness. Evaluation covers correctness-only, function-level Python generation on competitive programming benchmarks. Repository-level tasks (SWE-bench), non-Python languages, multi-step agentic tasks, and process reward metrics are outside the scope of this study.

6.0.3. 6.3 FUTURE WORK

Immediate: (1) Mechanism verification — re-run h-m2 with `max_new_tokens  $\in \{512, 1,024\}$` ; instrument GRPO training loop to capture live reward trajectories by difficulty bucket. (2) Full-scale evaluation — $4,449$ samples $\times 3$ epochs, saved checkpoints, bigcode-harness evaluation for the primary statistical gate.

Near-term: (3) Full RLEF-Binary training to convergence; reward distribution analysis stratified by APPS problem test-case count to distinguish cardinality bias from convergence equivalence as the primary null-result driver. (4) 1.3B model scale sanity check — a DeepSeek-Coder-1.3B training run was launched at the time of these experiments and was ongoing at report time; $\Delta_{ratio} \geq 1.0$ is the secondary gate.

Medium-term: (5) Weighted fraction reward (VerPO formulation, arXiv:2601.03525) to address the cardinality bias identified in h-m3. (6) Domain overlap analysis between APPS competition and LCB-Hard problems to distinguish difficulty-scaling from domain mismatch as the driver of the Δ pattern.

7. Conclusion

The APPS dataset contains reference solutions for 85.32% of competition-level problems. A DeepSeek-Coder-7B model trained on those solutions with supervised fine-tuning achieves 0.0 pass@1 on LiveCodeBench-Hard. The training data exists; the generalization does not. This observation revises the standard mechanistic account of why RLEF outperforms SFT at hard benchmarks: the relevant void is a

generalization void, not a dataset void.

Three contributions follow from this investigation. First, a controlled, reproducible, open-source pipeline for comparing RLEF against SFT across the full benchmark difficulty spectrum, using public components throughout. Second, empirical support for a statistically significant positive-ordered difficulty-scaling advantage for RLEF over SFT (JT $z=+56.10$, $p \approx 0$), with the largest gains at LiveCodeBench-Hard ($\Delta = +0.18$, smoke-scale proxy), all values pending full-scale confirmation. Third, a controlled null result on reward formulation: fraction-of-tests and binary RLEF rewards produce equivalent performance at APPS training scale ($p=0.552$), providing a third independent replication of reward formulation convergence under GRPO on a different model and dataset combination.

The generalization void finding has implications beyond this study. The common assumption that harder benchmark performance improves by adding more hard training examples may not hold if the model cannot generalize those examples to held-out problems. Understanding this generalization gap, and confirming whether RLEF’s self-generated feedback mechanism addresses it, is a central open question for the field.

References

Anonymous. VerPO: Verified reward policy optimization — weighted fractional reward for code RL. *arXiv preprint arXiv:2601.03525*, 2025.

Austin, J. et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

BigCode Evaluation Harness Contributors. BigCode evaluation harness. <https://github.com/bigcode-project/bigcode-evaluation-harness>, 2023.

Chen, M. et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Gehring, J. et al. RLEF: Grounding code language models in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.

Guo, D. et al. DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Hendrycks, D. et al. Measuring coding challenge competence with APPS. 34:23599–23612, 2021.

Jain, N. et al. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Jonckheere, A. R. A distribution-free k -sample test against ordered alternatives. *Biometrika*, 41(1/2): 133–145, 1954.

Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pp. 21314–21328, 2022.

Liu, J. et al. RLTF: Reinforcement learning from unit test feedback. *arXiv preprint arXiv:2307.04349*, 2023.

Majeed, S. A. et al. PPOCoder: Execution-based code generation using pretraining and ppo. In *Findings of the Association for Computational Linguistics: EACL*, 2023.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., and Guo, D. DeepSeek-Math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Terpstra, T. J. The asymptotic normality and consistency of Kendall’s test against trend, when ties are present in one ranking. *Indagationes Mathematicae*, 14:327–333, 1952.

von Werra, L. et al. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2024.

Wolf, T. et al. Transformers: State-of-the-art natural language processing, 2020.